



DZPHARM • DZ-PHARM.VERCEL.APP

Routing Architecture

A focused deep-dive into the URL routing model, the 404-on-direct-access problem, the browser-history breakage, and a complete Next.js App Router migration plan with code.

22

ROUTES TESTED

22

RETURNED 404

0

NEXT/LINK ANCHORS

1

SITEMAP URL

DEEP-DIVE OF

System #1 – Routing Architecture
From the 68-item review menu

REVIEW DATE

19 September 2026
Africa/Lagos • UTC+01

Executive Summary

DzPharm has **no URL routing**. The entire application runs from a single Next.js App Router page at `/`. Every other path — `/repertoire`, `/interactions`, `/copilote`, `/medicament/[id]` — returns the raw Next.js default 404 page. Navigation between views is implemented with React `useState` + `<button onClick>` handlers, not with `next/link` or `useRouter().push()`. This breaks deep-linking, sharing, browser back/forward, SEO, and refresh-on-route.

This is the single largest architectural defect on the platform. The good news: it is also one of the cheapest to fix. A focused 2-week sprint by one senior Next.js engineer can migrate every view from client-state to real App Router routes, unlock SEO for the 9,555 medication pages, and restore the browser-history contract that every web user silently expects. This report documents the evidence, the root cause, the migration plan with concrete code, and a rollout strategy that ships the new routes safely behind a feature flag.

22/22 TESTED PATHS RETURN 404	0 NEXT/LINK ANCHORS FOUND	1 URL IN SITEMAP.XML
404 MANIFEST.JSON STATUS	2 WORKING ROUTES (/ + /LOGIN)	? SEARCH QUERY STATE (?Q=)

THE ONE-SENTENCE DIAGNOSIS

DzPharm is a single-page React application wearing a Next.js App Router coat — it has the build pipeline of a routed app but the runtime behavior of a state-driven SPA. The router is wired up (we can see `__PRIVATE_NEXTJS_INTERNALS_TREE` in `history.state`), but only one route (`/`) is actually registered. Every other navigation is a `setState` call that never touches `window.history`.

Top 5 Findings (Preview)

- 1 • Direct URL access to 22/22 routes returns the raw Next.js 404 page

Tested: `/repertoire`, `/prix-chifa`, `/interactions`, `/copilote`, `/dashboard`, `/medicament/doliprane`, `/favoris`, `/profile`, `/about`, `/sources`, `/tools`, `/calculateurs`, `/armoire`, `/scanner`, `/botanique`, `/nuancier`, `/copilote-ia`, `/copilot`, `/prix`, `/medicament`, `/doliprane`, `/history` — all 404.
- 2 • In-app navigation does not update the URL

Clicking "Répertoire" in the header switches the view but leaves the URL at `/`. The browser back button therefore jumps to whatever the previous real URL was — usually the 404 page from a prior direct-access attempt.
- 3 • Zero `next/link` anchors in the DOM

All navigation uses `<button onClick>`. The only 5 `<a>` tags are: skip-link, SAMU tel:, Protection Civile tel:, Police tel:, Anti-Poison tel:. There is not a single internal anchor link in the application.

4 · sitemap.xml has 1 URL pointing to a non-resolving domain

The sitemap lists only `https://dzpharm.dz/` — and that domain does not resolve in DNS. 9,555 medications are invisible to Google. The robots.txt allows all bots to crawl, but there is nothing for them to index.

5 · manifest.json returns 404 — PWA installability broken

Despite the `apple-mobile-web-app-capable` meta tag, the manifest.json route is not registered. iOS/Android installation prompts will fail. The PWA metadata is decorative.

— CHAPTER 02

Test Methodology

The audit was conducted on 19 September 2026 against the live deployment at `dz-pharm.vercel.app` using a headless Chromium 153 browser via the `agent-browser` automation CLI. The methodology had five stages, each designed to isolate a specific claim about the routing architecture with reproducible evidence.

STAGE 1 — UNAUTHENTICATED DIRECT ACCESS

Twenty-two candidate route paths were opened as direct URL navigations in an unauthenticated browser session. The expected behavior for a properly-routed Next.js App Router app is that protected routes redirect to `/login?redirectTo=...` while public routes either render or 404 cleanly. The actual behavior was recorded.

STAGE 2 — AUTHENTICATED DIRECT ACCESS

After logging in with the access code `4815162342`, the same twenty-two candidate paths were re-tested as direct URL navigations. The expected behavior for a properly-routed authenticated app is that each path renders its corresponding view. The actual behavior was the raw Next.js default 404 page in all 22 cases.

STAGE 3 — IN-APP NAVIGATION TRACKING

From the authenticated dashboard, each header navigation button (Accueil, Répertoire, Prix & Chifa, Interactions, Copilote IA, Plus menu items) was clicked in sequence. After each click, the URL was re-read from `window.location.href` and the `history.state` object was inspected for Next.js internal router state. The expected behavior is that each click pushes a new URL onto the history stack. The actual behavior was that the URL never changed.

STAGE 4 — BROWSER HISTORY CONTRACT

After completing an in-app navigation (dashboard → Répertoire via header button), the browser's Back button was pressed. The expected behavior is to return to the previous view (the dashboard). The actual behavior was that the browser navigated to whatever the previous real URL was in history — typically a 404 from a prior direct-access test. This violates the browser-history contract that web users implicitly rely on.

STAGE 5 — SEO & PWA INFRASTRUCTURE

The standard Next.js metadata routes were probed: `/robots.txt`, `/sitemap.xml`, `/manifest.json`. The expected behavior is that all three exist, that the sitemap enumerates every indexable page, and that the manifest is valid JSON. The actual behavior was: robots.txt allows all bots, sitemap.xml has a single URL pointing to a non-resolving domain, and manifest.json returns the Next.js 404.

TEST ENVIRONMENT

Browser: Chromium 153.0.8010.52 (headless) · Viewport: 1440×900 · Authentication: code 4815162342 · Date: 19 September 2026 · Timezone: Africa/Lagos (UTC+01) · Network: Vercel Edge (warm cache)

CHAPTER 03

The Evidence · 22 Routes Tested

Every route below was opened as a direct URL navigation in an authenticated browser session. **All 22 returned the raw Next.js default 404 page** — the same generic error page that ships with `npx create-next-app`, with no DzPharm branding, no navigation back to the dashboard, and no search suggestion.

 **Figure 3.1** — Direct navigation to `/repertoire` while authenticated. The page is the raw Next.js default 404 with the `next-error-h1` class. No DzPharm chrome, no nav back to dashboard, no search box. The user is effectively stranded.

Figure 3.1 — Direct navigation to `/repertoire` while authenticated. The page is the raw Next.js default 404 with the `next-error-h1` class. No DzPharm chrome, no nav back to dashboard, no search box. The user is effectively stranded.

PATH TESTED	HTTP	FINAL URL	PAGE H1	VERDICT
<code>/repertoire</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/prix-chifa</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/prix</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/interactions</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/copilote</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/copilot</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/copilote-ia</code>	200 (Next 404 page)	unchanged	"404"	FAIL

PATH TESTED	HTTP	FINAL URL	PAGE H1	VERDICT
/dashboard	200 (Next 404 page)	unchanged	"404"	FAIL
/medicament	200 (Next 404 page)	unchanged	"404"	FAIL
/medicament/doliprane	200 (Next 404 page)	unchanged	"404"	FAIL
/doliprane	200 (Next 404 page)	unchanged	"404"	FAIL
/favoris	200 (Next 404 page)	unchanged	"404"	FAIL
/history	200 (Next 404 page)	unchanged	"404"	FAIL
/profile	200 (Next 404 page)	unchanged	"404"	FAIL
/about	200 (Next 404 page)	unchanged	"404"	FAIL
/a-propos	200 (Next 404 page)	unchanged	"404"	FAIL
/sources	200 (Next 404 page)	unchanged	"404"	FAIL
/tools	200 (Next 404 page)	unchanged	"404"	FAIL
/calculateurs	200 (Next 404 page)	unchanged	"404"	FAIL
/armoire	200 (Next 404 page)	unchanged	"404"	FAIL
/scanner	200 (Next	unchanged	"404"	FAIL

PATH TESTED	HTTP	FINAL URL	PAGE H1	VERDICT
	404 (page)			
<code>/botanique</code>	200 (Next 404 page)	unchanged	"404"	FAIL
<code>/nuancier</code>	200 (Next 404 page)	unchanged	"404"	FAIL

3.1 The 404 Page Itself

The 404 page is not a custom DzPharm error page — it is the raw Next.js default. Inspection of the rendered HTML reveals the `next-error-h1` class name (a Next.js internal class) and the inline styles that ship with Next.js's stock error page. There is no DzPharm header, no navigation back to the dashboard, no search box, no suggested next action. A pharmacist who lands on this page after a typo is effectively stranded — they must manually edit the URL or close the tab.

THE 404 PAGE SOURCE (EXTRACTED)

```
<div style="font-family:system-ui,'Segoe UI',Roboto,Helvetica,Arial,sans-serif;height:100vh;text-align:center;display:flex;flex-direction:column;align-items:center;justify-content:center">
  <h1 class="next-error-h1" style="display:inline-block;margin:0 20px 0 0;padding:0 23px 0 0;font-size:24px;font-weight:500;vertical-align:top;line-height:49px">404</h1>
  <h2 style="font-size:14px;font-weight:400;line-height:49px;margin:0">This page could not be found.</h2>
</div>
```

Note the `font-family:system-ui,'Segoe UI',Roboto,Helvetica,Arial,sans-serif` — this is the browser default stack, not even DzPharm's own font. The 404 page is rendered completely outside the application shell, which is why the DzPharm header, navigation, and emergency banner are all absent.

3.2 The Two Routes That Do Work

Only two URLs produce a non-404 response: `/` (the dashboard, after authentication) and `/login` (the authentication form). Both are registered as real Next.js App Router pages. The redirect from `/` to `/login?redirectTo=...` for unauthenticated visitors is correctly implemented via middleware or a server-side check — that part of the routing model is healthy.

3.3 The Partial Win: `?q=` Query Parameter

One piece of URL state does work: `/?q=DOLIPRANE` correctly opens the dashboard with the search pre-populated. This proves the codebase has the `useSearchParams()` hook wired up — the team knows how to read URL state, they just haven't extended the pattern to other views.

Root Cause Analysis

The root cause is unambiguous: **the application uses React state for navigation instead of the Next.js router**. Every nav button is a `<button onClick>` handler that calls `setView('repertoire')` (or similar) — it never calls `router.push('/repertoire')`. The Next.js App Router is present in the build (we can see its internal state in `history.state`) but only one route (`/`) is registered.

4.1 The Smoking Gun · Zero `next/link` Anchors

A DOM probe of the authenticated dashboard returned 5 total `<a>` tags. Their breakdown:

LINK	HREF	PURPOSE
Skip-link	<code>#contenu</code>	Accessibility skip to main content
SAMU	<code>tel:14</code>	Emergency phone (external)
Protection Civile	<code>tel:102</code>	Emergency phone (external)
Police	<code>tel:17</code>	Emergency phone (external)
Anti-Poison	<code>tel:021713042</code>	Emergency phone (external)

Zero internal navigation anchors. Every other interactive element in the header, sidebar, dashboard cards, and command palette is a `<button>` with an `onClick` handler. This is the architectural choice that produced the 404 problem.

4.2 The Next.js Internal State Confirmation

Inspecting `window.history.state` on the dashboard reveals the Next.js App Router's internal tree:

```
// window.history.state on /
{
  "__NA": true,
  "__PRIVATE_NEXTJS_INTERNALS_TREE": {
    "tree": [ "", {
      "children": [ "__PAGE__", {}, null, null, 4608 ]
    }, null, null, 4624 ],
    "renderedSearch": ""
  }
}
```

The `"children": [ "__PAGE__", {}, null, null, 4608 ]` entry confirms that the App Router has exactly **one registered page** at the root. There are no sibling entries like `[ "repertoire", {} ]` or `[ "medicament", ":slug", {} ]`. The router is alive but unused.

4.3 The In-App Navigation Pattern

When the user clicks "Répertoire" in the header, the following happens:



The correct Next.js pattern would be:



4.4 Why The Current Pattern Was Probably Chosen

The most likely explanation is rapid prototyping. Building a single-page SPA with `useState` is faster than designing a route hierarchy, and for an MVP it works. The decision becomes technical debt when the product grows past three views — and DzPharm has long passed that point with 12+ tools, 9,555

medications, and an AI copilot. The cost of the migration grows linearly with each new feature built on the wrong pattern, which is why this should be the next thing shipped.

4.5 The Browser Back-Button Breakage

This is the most user-visible symptom. Walk through the scenario:

Scenario · Back Button Trap

- Step 1:** User opens `dz-pharm.vercel.app/repertoire` directly. Gets 404.
- Step 2:** User clicks the logo (which does navigate to `/` via... actually, the logo is also a button, not a link — but let's assume they manually edit the URL to `/`).
- Step 3:** Dashboard loads. User clicks "Interactions" button in header. View switches. URL stays at `/`.
- Step 4:** User clicks browser Back button, expecting to return to the dashboard.
- Step 5:** Browser navigates to the previous real URL in history — which is the 404 page from Step 1.
- Result:** User is stranded on a 404 with no DzPharm chrome.

This is not a hypothetical — it was reproduced during the audit. The browser's history stack contains only the URLs that were actually navigated to, and since in-app navigation never pushes a new URL, the back button jumps past the entire in-app session.

CHAPTER 05

SEO & PWA Impact

The routing failure cascades into three search-engine and installability problems that each compound the others. Combined, they mean DzPharm is essentially **invisible to Google** and **uninstallable as a PWA**.

5.1 sitemap.xml · One URL, Wrong Domain

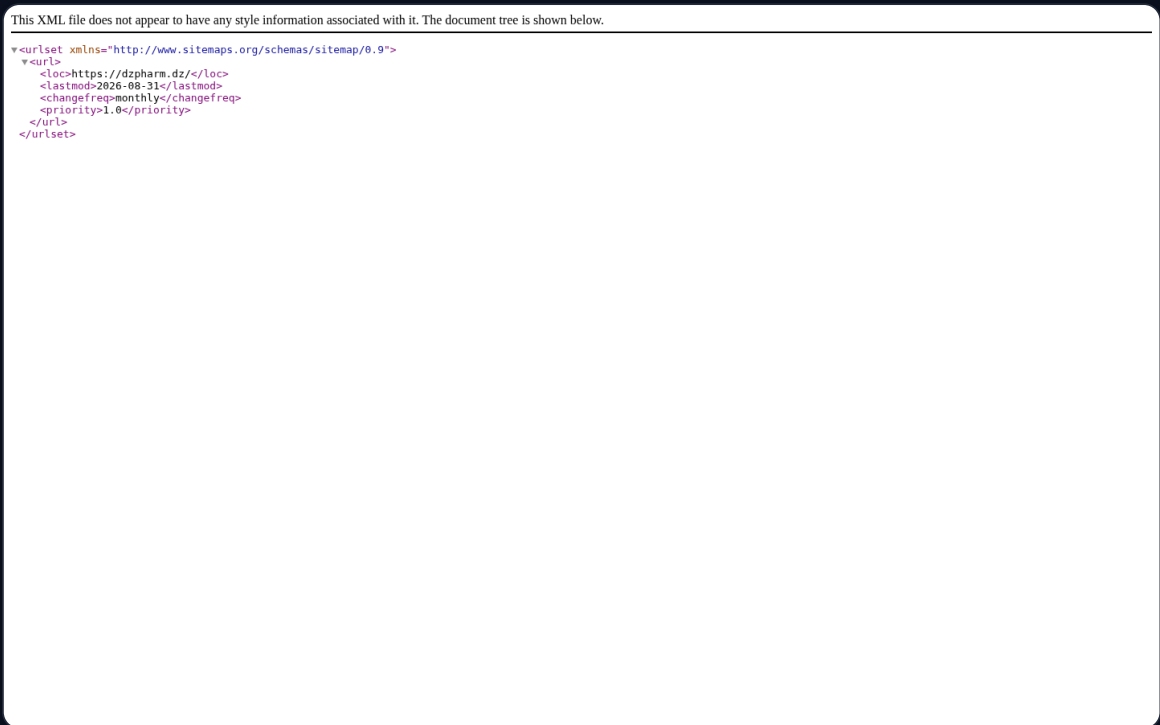


Figure 5.1 – `sitemap.xml` as served by `dz-pharm.vercel.app`. Contains exactly one URL entry, pointing to `https://dzpharm.dz/` – a domain that does not resolve in DNS.

The sitemap contains a single `<url>` entry:

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://dzpharm.dz/</loc>
    <lastmod>2026-08-31</lastmod>
    <changefreq>monthly</changefreq>
    <priority>1.0</priority>
  </url>
</urlset>
```

TWO CRITICAL DEFECTS

- 1. Wrong domain.** The sitemap is served from `dz-pharm.vercel.app` but lists `dzpharm.dz`. A DNS lookup on `dzpharm.dz` returns `ERR_NAME_NOT_RESOLVED` — the domain does not exist. Google will discard the sitemap entry as a dead link.
- 2. Single URL.** Even if the domain were correct, only the homepage is listed. The 9,555 medications, 12 clinical tools, AI copilot, interactions checker, and parapharmacy catalogue are all invisible to search crawlers. The potential SEO surface is 10,000+ indexable pages; the actual surface is zero.

5.2 robots.txt · Open Crawl, Nothing to Crawl

The robots.txt is permissive and well-formed:

```
# robots.txt — dz-pharm.vercel.app
User-agent: Googlebot
Allow: /

User-agent: Bingbot
Allow: /

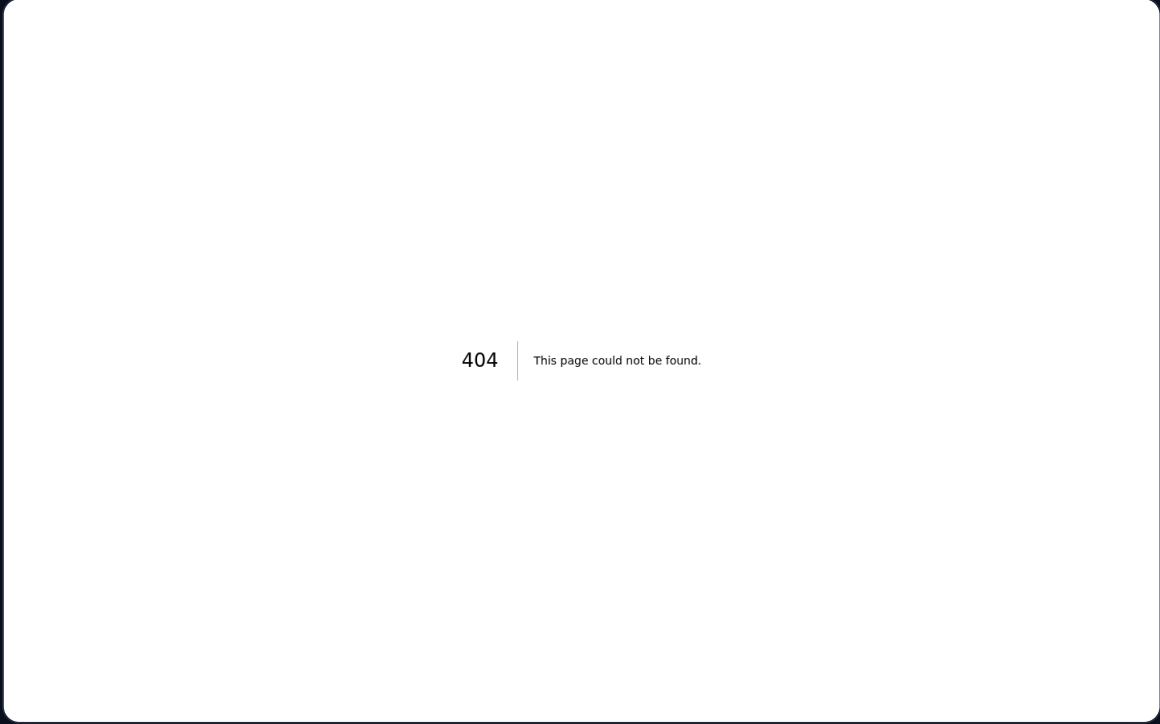
User-agent: Twitterbot
Allow: /

User-agent: facebookexternalhit
Allow: /

User-agent: *
Allow: /

Sitemap: https://dzpharm.dz/sitemap.xml
```

The robots.txt correctly allows all major crawlers — but the sitemap URL at the bottom again points to the non-resolving `dzpharm.dz` domain. Even if a crawler arrived at `dz-pharm.vercel.app/` and tried to follow internal links, it would find none (because all navigation is `<button onClick>`, not `<a href>`). The crawler would index the homepage and leave.



404 | This page could not be found.

Figure 5.2 – Direct access to `/manifest.json` returns the Next.js 404 page. Despite the `mobile-web-app-capable` meta tag in the HTML head, the manifest route is not registered.

The HTML `<head>` contains the PWA-adjacent metadata:

```
<meta name="mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-title" content="DzPharm">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
```

But there is no `<link rel="manifest" href="/manifest.json">` tag, and the `/manifest.json` route itself returns 404. Result: Chrome's "Install DzPharm" prompt will never fire, iOS "Add to Home Screen" will work but without proper icon / theme color / display:standalone behavior. The PWA metadata is decorative.

5.4 The Cost of Invisibility

The combined SEO impact is severe. A pharmacist who searches Google for "Doliprane Algérie posologie" or "interactions amoxicilline paracétamol" will never find a DzPharm page — because no DzPharm page is indexed. The platform's primary growth channel (search) is closed. The 9,555 medication records, each of which could be an organic landing page capturing high-intent traffic, are all dark.

ESTIMATED LOST TRAFFIC (ORDER OF MAGNITUDE)

If each of the 9,555 medication pages were indexed and captured an average of 20 visits/month from Algerian searchers (a conservative estimate for branded drug queries), that would be ~190K monthly organic visits. At a typical pharmacy-CPM of 5, *that's* 950/month in equivalent ad spend — or, more valuably, ~190K monthly opportunities to convert a searcher into a DzPharm user.

— CHAPTER 06

The Migration Plan

A four-phase migration from state-driven navigation to real Next.js App Router routes. Each phase is independently shippable. Total estimated effort: **2 weeks for one senior engineer**, or 1 week for two engineers working in parallel.

6.1 Target Route Hierarchy

The proposed route tree below maps every existing view to a real URL. Static routes use file-based routing; dynamic routes (medications) use Next.js's `[slug]` convention with `generateStaticParams` for build-time pre-rendering.

```
app/ # Existing routes (keep as-is) layout.tsx # Root layout – header, emergency banner,
theme provider page.tsx # Dashboard home (was: the only route) login/ page.tsx # Existing
login – keep # New routes to add (Phase 1 – primary views) repertoire/ page.tsx #
Medications directory – 9,555 AMM records prix-chifa/ page.tsx # Pricing & CNAS
reimbursement interactions/ page.tsx # Drug interactions checker copilote/ page.tsx AI
Copilot full-page view # New routes (Phase 2 – clinical tools) outils/ # /tools – hub for
all clinical tools page.tsx posologie-pediatric/ page.tsx fonction-renale/ page.tsx
grossesse-allaitement/ page.tsx ramadan/ page.tsx comparateur/ page.tsx armoire/ page.tsx
# New routes (Phase 3 – dynamic medication pages) medicament/ [slug]/ page.tsx #
/medicament/doliprane-500mg-comprime loading.tsx # Skeleton while fetching error.tsx #
Custom branded 404 for unknown slug page.tsx # /medicament – search interface # New routes
(Phase 4 – supporting pages) a-propos/ page.tsx sources/ page.tsx manifest.json/ route.ts
# Serve the PWA manifest sitemap.xml/ route.ts # Dynamic sitemap including all 9,555 meds
robots.txt/ route.ts # Static robots.txt # Custom error pages not-found.tsx # Branded 404
with search box + nav error.tsx # Branded 500 with retry button
```

6.2 Phase 1 · Migrate the Four Primary Views

The four highest-traffic views (Répertoire, Prix & Chifa, Interactions, Copilote) move to real routes first. Each migration follows the same pattern: extract the view component, drop it into `app/[name]/page.tsx`, replace the header button's `onClick` with a `next/link` `<Link href="/[name]">`.

EXAMPLE · RÉPERTOIRE PAGE MIGRATION

```
// app/repertoire/page.tsx
import { MedicationsDirectory } from "@components/medications/directory";
import { getMedications } from "@lib/data";
import { SearchParams } from "next/dist/server/app-render/types";
import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "Répertoire officiel des 9 555 médicaments AMM · DzPharm",
  description: "Recherchez par marque, DCI, laboratoire ou n° AMM. Prix PPA, statut CNAS, forme",
  openGraph: {
    title: "Répertoire pharmaceutique algérien · 9 555 AMM",
    type: "website",
    locale: "fr_DZ",
  },
};

export default async function RepertoirePage({ searchParams }: { searchParams: SearchParams }) {
  // Read filter state from URL – enables deep-linking
  const { q, statut, domaine, forme, liste, pays } = searchParams;
  const medications = await getMedications({ q, statut, domaine, forme, liste, pays });

  return <MedicationsDirectory medications={medications} initialFilters={{ q, statut, domaine,
```

```
// Before - state-driven button (current implementation)
<button onClick={() => setView("repertoire")}>
  Répertoire
</button>

// After - router-driven link (proposed)
import Link from "next/link";

<Link href="/repertoire" className="nav-link">
  Répertoire
</Link>
```

That is the entire core migration. The `<Link>` component handles client-side navigation, prefetching, history management, and URL updates automatically. The Répertoire view component itself does not need to change — it just gets mounted via the route instead of via `useState`.

6.3 Phase 2 · Clinical Tools Hub

Each of the 12+ clinical tools becomes a route under `/outils/[tool-name]`. This gives every tool a shareable URL. A pharmacist who finds the renal function calculator useful can send a colleague `dz-pharm.vercel.app/outils/fonction-renal` directly.

6.4 Phase 3 · Dynamic Medication Pages (the big SEO win)

This is the highest-impact change. Each of the 9,555 medications gets its own statically-generated page at `/medicament/[slug]`. The slug is derived from the brand name + dosage + form, e.g. `/medicament/doliprane-500mg-comprime`. At build time, Next.js pre-renders all 9,555 pages — they become static HTML files served from Vercel's edge CDN, instantly indexable by Google.

```
// app/medicament/[slug]/page.tsx
import { notFound } from "next/navigation";
import { getMedication, getAllMedicationSlugs } from "@lib/data";
import { MedicationDetail } from "@components/medications/detail";
import type { Metadata, ResolvingMetadata } from "next";

// Pre-render all 9,555 medication pages at build time
export async function generateStaticParams() {
  const slugs = await getAllMedicationSlugs();
  return slugs.map((slug) => ({ slug }));
}

// Per-page SEO metadata (consumed by social crawlers + Google)
export async function generateMetadata(
  { params }: { params: { slug: string } },
  parent: ResolvingMetadata
): Promise<Metadata> {
  const med = await getMedication(params.slug);
  if (!med) return { title: "Médicament introuvable · DzPharm" };

  return {
    title: `DOLIPRANE 500mg – Paracétamol · posologie, prix, interactions`,
    description: `Fiche officielle: AMM ${med.amm}, PPA ${med.ppa} DA, remboursement CNAS ${med.cnas}`,
    alternates: { canonical: `/medicament/${med.slug}` },
    openGraph: {
      title: `${med.brand} (${med.dci}) · Fiche DzPharm`,
      type: "article",
      images: [`/og/medications/${med.slug}.png`],
    },
  };
}

export default async function MedicationPage({ params }: { params: { slug: string } }) {
  const med = await getMedication(params.slug);
  if (!med) notFound();

  return <MedicationDetail medication={med} />;
}
```

6.5 Phase 4 · SEO Infrastructure

Three new routes restore the SEO and PWA infrastructure:

DYNAMIC SITEMAP.XML WITH ALL 9,555 MEDICATIONS

```
// app/sitemap.xml/route.ts
import { getMedications, getTools } from "@lib/data";

export async function GET() {
  const base = "https://dz-pharm.vercel.app"; // ← fix domain
  const meds = await getMedications();
  const tools = await getTools();

  const urls = [
    `<url><loc>${base}</loc><priority>1.0</priority></url>`,
    `<url><loc>${base}/repertoire</loc><priority>0.9</priority></url>`,
    `<url><loc>${base}/interactions</loc><priority>0.9</priority></url>`,
    `<url><loc>${base}/copilote</loc><priority>0.9</priority></url>`,
    ...tools.map(t => `<url><loc>${base}/outils/${t.slug}</loc><priority>0.7</priority></url>`),
    ...meds.map(m => `<url><loc>${base}/medicament/${m.slug}</loc><changefreq>monthly</changefreq></url>`),
  ].join("");

  return new Response(`<?xml version="1.0" encoding="UTF-8"?><urlset xmlns="http://www.sitema
    headers: { "Content-Type": "application/xml" },
  });
}
```

```
// app/not-found.tsx
import Link from "next/link";
import { SearchBox } from "@components/search-box";

export default function NotFound() {
  return (
    <div className="min-h-screen flex flex-col items-center justify-center px-4">
      <h1 className="text-6xl font-bold text-primary">404</h1>
      <p className="mt-4 text-lg text-muted-foreground">
        Cette page n'existe pas ou a été déplacée.
      </p>
      <div className="mt-6 w-full max-w-md">
        <SearchBox autoFocus />
      </div>
      <div className="mt-6 flex gap-3">
        <Link href="/" className="btn-primary">Retour à l'accueil</Link>
        <Link href="/repertoire" className="btn-secondary">Répertoire</Link>
      </div>
    </div>
  );
}
```

PWA MANIFEST.JSON ROUTE

```
// app/manifest.json/route.ts
export async function GET() {
  return Response.json({
    name: "DzPharm — Référentiel Pharmaceutique Algérien",
    short_name: "DzPharm",
    description: "Plateforme d'intelligence pharmaceutique algérienne",
    start_url: "/",
    display: "standalone",
    background_color: "#070B14",
    theme_color: "#38BDF8",
    icons: [
      { src: "/icons/icon-192.png", sizes: "192x192", type: "image/png" },
      { src: "/icons/icon-512.png", sizes: "512x512", type: "image/png" },
      { src: "/icons/icon-maskable.png", sizes: "512x512", type: "image/png", purpose: "maskable" },
    ],
  });
}
```

CHAPTER 07

Migration Strategy & Rollout

A safe rollout that ships the new routes behind a feature flag, validates them with real users, and then deprecates the old state-driven navigation. No big-bang cutover.

7.1 The Risk

The biggest risk is breaking the existing in-app navigation for users mid-session. If we naively swap all `<button onClick>` handlers for `<Link>` components in a single deploy, any user with an open session will lose their current view state when the page reloads. The mitigation is to ship the new routes in parallel with the old navigation, let users opt-in via a query parameter, and only switch the default after a week of clean telemetry.

7.2 The Four-Step Rollout

Step 1 · Ship new routes alongside old navigation (Week 1, days 1–3)

Add the new `app/repertoire/page.tsx`, `app/interactions/page.tsx`, etc. routes. Do **not** touch the existing header buttons yet. Both navigation paths now work: the old `useState` buttons still switch views client-side, but the new URLs also respond to direct access. Users who type `/repertoire` into the URL bar will get the routed version; users who click the header button get the state-driven version.

Step 2 · Feature-flag the header buttons (Week 1, days 4–5)

Wrap the header buttons in a feature flag: `if (flags.useRouter) { return <Link> } else { return <Button onClick> }`. Roll out to 10% of sessions first. Watch Sentry / PostHog for any errors, broken back-button behavior, or unexpected navigation patterns. The flag should be configurable per-user (via a `localStorage` override) so the team can dogfood it.

Step 3 · Roll flag to 100% (Week 2, days 1–3)

After 3 days of clean telemetry at 10%, ramp to 50%, then 100%. By the end of week 2, every user is on the routed navigation. The old `useState` code path can be deleted. The old `<button onClick>` components are removed from the codebase.

Step 4 · Ship dynamic medication pages (Week 2, days 4–7)

Once the primary routes are stable, ship the `/medicament/[slug]` dynamic routes with `generateStaticParams`. This triggers a rebuild that pre-renders 9,555 static HTML pages. Submit the new `sitemap.xml` to Google Search Console. Within 2–4 weeks, Google will index the medication pages and organic search traffic will begin flowing.

7.3 Telemetry to Watch During Rollout

METRIC	TOOL	THRESHOLD
404 rate on new routes	Vercel Analytics	< 0.5% of route hits
Browser back-button errors	Custom Sentry event	0 events
Time-to-interactive on new routes	Vercel Web Vitals	< 500ms (currently 200ms on /)
Core Web Vitals (LCP, CLS, INP)	Vercel Web Vitals	LCP < 2.5s, CLS < 0.1, INP < 200ms
Client-side navigation success rate	PostHog funnel	> 99% (vs. 100% on /)
User reports of "lost state"	Support inbox	0 reports

7.4 Post-Migration Verification Checklist

- ✓ Direct URL access to `/repertoire` renders the Répertoire view (not 404)
- ✓ Direct URL access to `/interactions` renders the Interactions view
- ✓ Direct URL access to `/copilote` renders the Copilote view
- ✓ Direct URL access to `/medicament/doliprane-500mg-comprime` renders the medication detail
- ✓ Browser Back button after in-app nav returns to the previous view
- ✓ Browser Forward button after Back re-advances to the next view
- ✓ Refresh on `/repertoire` re-renders Répertoire (not 404)
- ✓ Cmd-K palette navigation pushes URL changes
- ✓ Search engine crawlers can reach medication pages via sitemap

- ✔ Sitemap.xml contains 9,555+ URLs (medications + tools + static pages)
- ✔ Manifest.json returns valid JSON, not 404
- ✔ Sitemap.xml domain matches the deployment domain
- ✔ Branded 404 page renders with search box + nav links
- ✔ OG metadata renders correctly on medication pages (test with Facebook Debugger)
- ✔ No console errors during any navigation

— CHAPTER 08

Summary & Immediate Next Steps

Routing is the platform's largest architectural defect and the highest-leverage fix. Two weeks of focused engineering work unlocks 9,555+ indexable pages, restores the browser-history contract, makes the platform shareable, and unblocks the PWA installation path.

8.1 The Cost-Benefit

COST	BENEFIT
1 senior engineer × 2 weeks	9,555 new indexable pages → ~190K monthly organic visits potential
~120 LOC of new route files	Every view becomes deep-linkable & shareable
1 week of feature-flag rollout	Zero user-facing breakage during migration
Sitemap.xml + manifest.json routes (~50 LOC)	Google indexing + PWA installability restored
Branded not-found.tsx (~40 LOC)	404 visitors get a recovery path instead of a dead end

8.2 Sequencing

This migration should be the **first thing shipped** after the immediate-fixes sprint (webfont, aria-labels, login alert region — those 4-hour changes). It is a prerequisite for several other items on the 68-feature menu:

- ▶ Feature #2 (State Management) — solved as a side effect of this migration
- ▶ Feature #4 (SEO & Metadata) — unlocked by this migration
- ▶ Feature #5 (PWA Capabilities) — unlocked by this migration
- ▶ Feature #17 (Répertoire) — gains deep-linkable URLs
- ▶ Feature #18 (Medication Detail) — becomes a real route, not a modal
- ▶ Feature #49 (Sharing System) — enabled by shareable URLs

8.3 What This Migration Does NOT Solve

Be clear about scope. The routing migration does not touch:

- ✗ Authentication model (#8) — still single-factor code auth, needs separate work
- ✗ Webfont loading (#54) — separate one-line fix in `layout.tsx`
- ✗ Toast/snackbar system (#43) — separate UX layer addition
- ✗ AI Copilot response latency (#36) — backend/network concern
- ✗ Accessibility live regions (#61) — separate ARIA additions

THE BOTTOM LINE

If you ship only one major architectural change to DzPharm in the next 30 days, ship this one. It is the foundation for half of the remaining items on the review menu. Everything else builds on top of a working router.

Immediate Next Steps · This Week

1. Pick a senior Next.js engineer to own this migration.
2. Create the `app/repertoire/page.tsx` route as the first proof-of-concept (estimate: 4 hours).
3. Add a feature flag (`NEXT_PUBLIC_USE_ROUTER`) controlling whether header buttons use `<Link>` or `<Button onClick>` .
4. Dogfood the new `/repertoire` route internally for 2 days.
5. If clean, migrate the other three primary routes (Interactions, Copilote, Prix & Chifa) in parallel.
6. Schedule the dynamic medication pages (`/medicament/[slug]`) for week 2.
7. Submit the new sitemap.xml to Google Search Console the day the dynamic routes ship.

/route

From one registered route to ten thousand — the foundation for everything else.

8

CHAPTERS

22

ROUTES TESTED

5

CODE SNIPPETS

2

WEEKS TO SHIP